

Questão 05 - Modernizar deployment legado

Prompt (aplicando B-A-B)

[BEFORE]

Este é o Deployment em produção do Chronos (API gateway core da empresa),
escrito há 3 anos e nunca revisado desde então:

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: chronos-api
```

```
  namespace: production
```

```
spec:
```

```
  replicas: 1
```

```
  selector:
```

```
    matchLabels:
```

```
      app: chronos-api
```

```
  template:
```

```
    metadata:
```

```
      labels:
```

```
        app: chronos-api
```

```
    spec:
```

```
      containers:
```

```
        - name: api
```

```
          image: chronos-api:latest
```

```
          ports:
```

```
            - containerPort: 8080
```

```
          env:
```

```
            - name: DB_PASSWORD
```

```
              value: "<valor em texto plano no manifest>"
```

```
            - name: JWT_SECRET
```

value: "<valor em texto plano no manifest>"

Problemas identificados neste estado atual:

- replicas: 1 → sem alta disponibilidade, ponto único de falha
- image: chronos-api:latest → tag não versionada, deploys não são reproduzíveis nem reversíveis com segurança
- Credenciais sensíveis (DB_PASSWORD, JWT_SECRET) hardcoded em texto plano diretamente no manifest, visível para qualquer um com acesso de leitura ao YAML
- Nenhum resource request/limit definido → risco de um pod consumir recursos do node inteiro e afetar outros workloads
- Nenhuma liveness/readiness probe → Kubernetes não sabe identificar um pod travado ou ainda não pronto para receber tráfego
- Nenhum securityContext → container roda como root por padrão

[AFTER]

O manifest modernizado precisa atender ao padrão de produção atual da Hill Valley Tech, aprovado por Strickland (segurança/compliance) e Lorraine (SRE), contendo:

- Alta disponibilidade (múltiplas réplicas)
- Imagem com tag versionada semântica (nunca "latest")
- Secrets referenciados via Kubernetes Secret (secretKeyRef), nunca em texto plano no manifest
- Resource requests e limits definidos para CPU e memória
- Liveness probe e readiness probe configuradas
- securityContext non-root, com demais hardenings recomendados (readOnlyRootFilesystem quando aplicável, drop de capabilities)
- Demais boas práticas de produção padrão da empresa (ex.: terminationGracePeriodSeconds, estratégia de rollout)

[BRIDGE]

Reescreva o Deployment do Chronos partindo do manifest atual (Before) até

o estado desejado (After), aplicando cada uma das mudanças necessárias.

Para os secrets, crie também o manifest de Secret correspondente (referenciado, com valores como placeholder — não invente ou reutilize os valores reais expostos), a ser aplicado separadamente e nunca versionado no mesmo lugar que o código-fonte. Assuma que a aplicação expõe endpoints /healthz (liveness) e /ready (readiness) na porta 8080. Mantenha o nome do Deployment e a porta do container.

[FORMATO DE SAÍDA]

1. O YAML do Secret (com valores placeholder)
2. O YAML do Deployment modernizado
3. Uma tabela resumo: Antes | Depois | Motivo da mudança

Modelo de IA utilizado

Claude Sonnet 4.5 (Anthropic)

Justificativa do uso

Modelo de referência do curso. Forte aderência a prompts estruturados em múltiplos blocos, raciocínio encadeado consistente e geração de artefatos técnicos (YAML, HCL, bash, SQL) com alto grau de precisão. Escolhido como baseline por permitir avaliação consistente dos frameworks ao longo de todo o desafio.

Output gerado

1. Secret (aplicado separadamente, fora do versionamento do Deployment):

apiVersion: v1

kind: Secret

metadata:

name: chronos-api-secrets

namespace: production

type: Opaque

stringData:

DB_PASSWORD: "<definido via pipeline/Secrets Manager, não commitado>"

JWT_SECRET: "<definido via pipeline/Secrets Manager, não commitado>"

2. Deployment modernizado:

apiVersion: apps/v1

kind: Deployment

metadata:

name: chronos-api

namespace: production

spec:

replicas: 3 # HA: elimina ponto único de falha

selector:

matchLabels:

app: chronos-api

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxUnavailable: 1
    maxSurge: 1
template:
  metadata:
    labels:
      app: chronos-api
  spec:
    terminationGracePeriodSeconds: 30
    securityContext:
      runAsNonRoot: true
      runAsUser: 1000
      fsGroup: 1000
    containers:
      - name: api
        image: chronos-api:1.4.2    # imagem versionada, nunca "latest"
        ports:
          - containerPort: 8080
        env:
          - name: DB_PASSWORD
            valueFrom:
              secretKeyRef:
                name: chronos-api-secrets
                key: DB_PASSWORD
          - name: JWT_SECRET
            valueFrom:
              secretKeyRef:
                name: chronos-api-secrets
                key: JWT_SECRET
```

resources:

requests:

cpu: "250m"

memory: "256Mi"

limits:

cpu: "500m"

memory: "512Mi"

livenessProbe:

httpGet:

path: /healthz

port: 8080

initialDelaySeconds: 10

periodSeconds: 15

timeoutSeconds: 3

failureThreshold: 3

readinessProbe:

httpGet:

path: /ready

port: 8080

initialDelaySeconds: 5

periodSeconds: 10

timeoutSeconds: 3

failureThreshold: 3

securityContext:

allowPrivilegeEscalation: false

readOnlyRootFilesystem: true

capabilities:

drop:

- ALL

3. Tabela resumo:

Antes	Depois	Motivo da mudança
replicas: 1	replicas: 3 + RollingUpdate	Alta disponibilidade; elimina downtime em deploys e falhas de nó
image: chronos-api:latest	image: chronos-api:1.4.2	Reprodutibilidade e rollback seguro; "latest" pode mudar sem controle
Secrets em texto plano no env	secretKeyRef apontando pra Secret separado	Credenciais fora do manifest versionado, alinhado a Strickland/compliance
Sem resources	requests/limits de CPU e memória	Evita "noisy neighbor" e uso descontrolado de recursos no cluster
Sem probes	livenessProbe + readinessProbe	Kubernetes consegue detectar travamentos e só rotear tráfego a pods prontos
Sem securityContext	runAsNonRoot, drop: ALL, readOnlyRootFilesystem	Reduz superfície de ataque; container não roda como root
Sem grace period	terminationGracePeriodSeconds: 30	Encerramento gracioso de conexões em rolling updates

Justificativa — como B-A-B aparece no prompt

Componente	Onde aparece	Por quê
Before	Manifest original colado no prompt + lista explícita dos problemas (réplica única, latest, secrets expostos, sem probes, sem securityContext)	Ancora a IA no estado real e problemático, dando contexto suficiente para que ela entenda por que cada mudança é necessária, não apenas o que mudar
After	Lista dos requisitos do padrão de produção da empresa (HA, versionamento, secrets externos, probes, non-root, etc.)	Define claramente a "meta" — o estado final que precisa ser alcançado, servindo como checklist de validação do output
Bridge	"Reescreva o Deployment... aplicando cada uma das mudanças necessárias... crie também o Secret... assuma que a aplicação expõe /healthz e /ready"	É a instrução de execução propriamente dita — conecta o estado atual ao desejado com ações concretas, incluindo pressupostos necessários (endpoints de probe) que não estavam no manifest original